

Plan de monitoring — Pipeline ETL

CheckItAI

Étape 5 — stratégie de surveillance du pipeline en production

Ce document décrit comment surveiller le pipeline d'acquisition de données multimodales une fois en production : quels indicateurs suivre, à quels seuils déclencher une alerte, comment gérer les erreurs, et à quelle fréquence vérifier. Il est cohérent avec les automatisations déjà en place (DAG Airflow `checkit_etl`) et avec le tableau de bord KPI.

1. Objectif

Garantir que le pipeline produit des données **fiables** (texte + image valides), **à temps**, et **sans coût excessif** (quota API). Le monitoring doit permettre de détecter rapidement une dégradation et d'alerter la bonne personne.

2. Indicateurs surveillés et seuils d'alerte

Les seuils sont tirés des KPI du tableau de bord (`src/dashboard_kpi.py`).

Indicateur	Mesure	Seuil d'alerte	Gravité
Taux de validité	% d'entrées avec texte + image valides	< 80 %	Élevée
Volume extrait	nb d'articles récupérés par run	0 article	Critique
Durée du run	durée totale extract + transform + load	> 5 min	Moyenne
Échec de tâche	état d'une tâche Airflow	état <code>failed</code>	Critique
Quota API	appels consommés / quota NewsData.io	> 90 % du quota	Moyenne

Justification des seuils : un taux de validité qui chute signale un changement côté source (champ image manquant, format modifié). Un volume nul ou un échec de tâche bloquent l'alimentation du moteur et sont donc critiques. Le seuil de durée et de quota protègent respectivement la fraîcheur des données et le budget API.

3. Gestion des erreurs

La gestion est en partie **déjà automatisée** dans le code et le DAG :

- **Au niveau du code** : chaque appel réseau et écriture fichier est encadré par `try/except` avec journalisation (`logging`). L'erreur HTTP 429 (quota dépassé) est identifiée explicitement.
- **Au niveau d'Airflow** :
 - **Retries** : `retries=2` et `retry_delay` (1 min) sont définis sur les tâches du DAG (via `default_args`) pour absorber les incidents réseau transitoires.
 - **Dépendances** : `transform` ne s'exécute que si `extract` a réussi, et `load` que si `transform` a réussi (enchaînement du DAG). Une erreur stoppe la chaîne.
 - **Alertes** : `email_on_failure` ou un callback (`on_failure_callback`) vers une notification (e-mail / Slack) à chaque échec de tâche.
- **Données invalides** : les entrées sans texte ou sans image sont écartées par le pipeline (elles ne polluent pas la base), mais comptabilisées dans le taux de validité.

4. Fréquence des vérifications

Vérification	Fréquence	Moyen
État des runs du DAG	À chaque exécution	Alerte automatique Airflow (échec)
Tableau de bord KPI	Quotidienne	Consultation Streamlit
Taux de validité / volume	Hebdomadaire	Revue des KPI sur la semaine
Quota API	Mensuelle	Suivi de la consommation NewsData.io

La fréquence des runs du DAG dépendra du besoin métier (ex. exécution quotidienne). Le monitoring automatique (alertes Airflow) couvre l'urgent ; les revues périodiques couvrent les tendances de fond.

5. Rôles et destinataires des alertes

- **Alerte critique** (échec, volume nul) → notification immédiate à l'ingénieur data d'astreinte.
- **Alerte moyenne** (durée, quota) → e-mail à l'équipe data, traitement sous 24 h.
- **Tendances** (taux de validité) → revue en réunion d'équipe hebdomadaire.

6. Cohérence avec les automatisations existantes

Ce plan s'appuie sur ce qui est déjà construit :

- le **DAG Airflow** fournit l'historique des runs et la base des alertes d'échec ;
- le **tableau de bord Streamlit** fournit la lecture des KPI pour les revues ;
- les **logs** des scripts (étapes 2 et 3) tracent le détail de chaque transformation.

Aucune brique supplémentaire n'est nécessaire pour démarrer le monitoring : il s'agit d'activer les alertes Airflow et d'inscrire les revues KPI dans le rythme de l'équipe.